In [1]:

```
"""
データまたはデータカタログのタイトル・説明文から地理空間情報の
geonamesの候補を推定するための関数群を定義
"""
```

Out[1]:

'¥nデータまたはデータカタログのタイトル・説明文から地理空間情報の¥ngeonamesの候補
を推定するための関数群を定義¥n'

In [2]:

```
# Modules
import geocoder
import requests
import json
import pandas as pd
import chardet

# geocoderのgeoname関連マニュアル
# https://geocoder.readthedocs.io/providers/GeoNames.html

# 独自モジュール
import pick_locationname
```

In [3]:

```
# 各種パラメータの設定
GEONAME_USER_KEY = 'jsugawa'
GEONAME_MAXROWS = 10
GEONAME_LANG = 'ja'
GEONAME_COUNTRY = 'JP'
GEONAME_FEATURECLASS = ['A','P']
GEONAME_GETFEATURE_URL = 'http://api.geonames.org/getJSON'
GEONAME_FINDNEARBY_URL = 'http://api.geonames.org/findNearbyJSON'
GEONAME_FINDNEARBY_RADIUS = 20
```

In [4]:

```
# 関数の定義

# テキストから地名のキーワードを決める関数を定義(現状は自治体名、都道府県名のみ抽出可能)
def extract_LocationKeywords(text):
    keyword_lg_list = pick_locationname.included_localgovs(text)
    keyword_pf_list = pick_locationname.included_prefectures(text)

    if len(keyword_lg_list) > 0:
        keyword_list = keyword_lg_list
    elif len(keyword_pf_list) > 0:
        keyword_list = keyword_pf_list
    else:
        keyword_list = []

    return keyword_list


# ファイルから緯度経度の一覧を取得する
def extract_latlngArray_csv(filepath):
    """
    CSVファイルを読み込んで緯度経度のarrayを出力する
    """
    # CSVファイルをDataframeとして読み込む
    try:
        df = pd.read_csv(filepath,encoding='shift_jis')
    except UnicodeDecodeError:
        try :
            df = pd.read_csv(filepath,encoding='utf-8')
        except UnicodeDecodeError:
            print("Error in read_csv")
            return None

    #with open(sample_file_path, "rb") as f:
    #    encoding_sample = chardet.detect(f.read())
    #    print(encoding_sample)

    if ('緯度' in df.columns) and ('経度' in df.columns):
        # 緯度または経度に欠損がある行は削除
        df = df.dropna(subset=['緯度', '経度'])

        # 緯度、経度のnumpy arrayを取得

        lat_array = df['緯度'].values
        lng_array = df['経度'].values

        return lat_array, lng_array
    elif ('latitude' in df.columns) and ('longitude' in df.columns):

        # 緯度または経度に欠損がある行は削除
        df = df.dropna(subset=['latitude', 'longitude'])

        # 緯度、経度のnumpy arrayを取得

        lat_array = df['latitude'].values
        lng_array = df['longitude'].values

        return lat_array, lng_array
    else:
        print("Error: 緯度、経度の列がありません")
```

```
        return None

# ファイルから緯度経度のBoundingboxを取得する
def extract_latlngBbox_csv(filepath):

    lat_array, lng_array = extract_latlngArray_csv(filepath)
    east = lng_array.max()
    west = lng_array.min()
    north = lat_array.max()
    south = lat_array.min()

    return east,west,north,south


# 地名キーワードからgeonameを検索する関数を定義
def FindByKeyword(
    locationKeyword,
    key=GEONAME_USER_KEY,
    maxRows=GEONAME_MAXROWS,
    featureClass=[],
    lang=GEONAME_LANG,
    country=GEONAME_COUNTRY):
    """
    地名のキーワードを指定して、候補となるgeonameのリストを出力
    """


    g = geocoder.geonames(
        location=locationKeyword,
        key=key,
        maxRows=maxRows,
        featureClass=featureClass,
        lang=lang,
        country=country,
        )

    return g


# geoname idから属性を調べる関数を定義
# (geocoderのmethod=detail指定だと結果が英語のみであり、手動で作成)
"""
g2 = geocoder.geonames(
    1849896,
    method='details',
    key=GEONAME_USER_KEY,
    lang=GEONAME_LANG,
    )

print(
    g2.geonames_id,
    g2.address,
    g2.country,
    g2.state,
    g2.lat,
    g2.lng,
    g2.feature_class,
    g2.admin2,
    g2.admin3,
    g2.bbox,
)
"""
```

```
def getFeautures(geonameId, username=GEONAME_USER_KEY, lang=GEONAME_LANG):
    """
    指定したgeonameIdの地名の属性値を辞書形式で出力する
    """
    params ={
        'geonameId':geonameId,
        'username':username,
        'lang':lang,
    }

    r = requests.get(
        url=GEONAME_GETFEATURE_URL,
        params=params
        )

    #print('request query: ',r.url)
    #print('status code: ', r.status_code)

    result_dict = json.loads(r.text)
    return result_dict



# 緯度経度のBoundingBox範囲から検索する
# (FeatureClass='A' or 'P'のみ抽出)

def FindbyBbox(east, west, north, south):
    g = geocoder.geonames(
        '',
        key=GEONAME_USER_KEY,
        maxRows=GEONAME_MAXROWS,
        featureClass=GEONAME_FEATURECLASS,
        lang=GEONAME_LANG,
        country=GEONAME_COUNTRY,
        east=east,
        west=west,
        north=north,
        south=south,
        )

    return g

'''
g4 = geocoder.geonames(
    location='',
    proximity=[35.70552,139.46125],
    key=GEONAME_USER_KEY,
    maxRows=GEONAME_MAXROWS,
    featureClass=['A','P'],
    lang=GEONAME_LANG,
    country=GEONAME_COUNTRY,
#    east=139.6,
#    west=139.5,
#    north=35.41,
#    south=35.40,
    )

for elem in g4:
    print(
        elem.geonames_id,
```

```
        elem.address,
        elem.country,
        elem.state,
        elem.feature_class,
        )
'''


# 手動でFindNearby用の関数を定義(geocoderのProximity指定だと結果がいまいちだったため)
def FindNearby(lat, lng, username=GEONAME_USER_KEY, radius=GEONAME_FINDNEARBY_RADIUS, maxRows=GE
ONAME_MAXROWS, lang=GEONAME_LANG, country=GEONAME_COUNTRY):
    """
    指定した緯度、経度の近くの地名をリスト形式で出力する
    """
    params ={
        'lat':lat,
        'lng':lng,
        'username':username,
        'radius':radius,
        'maxRows':maxRows,
        'lang':lang,
        'country':country,
    }

    r = requests.get(
        url=GEONAME_FINDNEARBY_URL,
        params=params
        )

    # print('request query: ',r.url)
    # print('status code: ', r.status_code)

    result_dict = json.loads(r.text)
    return result_dict['geonames']
```

In [5]:

```
# 関数の動作テスト
if __name__ == '__main__':
    # 関数extract_LocationKeywordsのテスト1
    print("==== extract_LocationKeywordsのテスト1 ====")
    title_text = "【蕨市】避難施設一覧"
    description_text = "蕨市内の避難施設の一覧です。"
    # https://opendata.pref.saitama.lg.jp/data/dataset/c-users-2348-desktop
    print("title:", title_text)
    print("description:", description_text)
    print("Keywords:",extract_LocationKeywords(title_text + description_text))

    # 関数extract_LocationKeywordsのテスト2
    print("==== extract_LocationKeywordsのテスト2 ====")
    title_text = "【埼玉県】公共施設情報"
    description_text = "埼玉県が保有する公共施設の住所、連絡先及び位置情報等のデータ"
    # https://opendata.pref.saitama.lg.jp/data/dataset/a2290sisetu
    print("title:", title_text)
    print("description:", description_text)
    print("Keywords:",extract_LocationKeywords(title_text + description_text))
    print("¥n")
```

```
==== extract_LocationKeywordsのテスト1 ====
title: 【蕨市】避難施設一覧
description: 蕨市内の避難施設の一覧です。
Keywords: ['蕨市']
==== extract_LocationKeywordsのテスト2 ====
title: 【埼玉県】公共施設情報
description: 埼玉県が保有する公共施設の住所、連絡先及び位置情報等のデータ
Keywords: ['埼玉県']
```

In [6]:

```
    # extract_latlngArray_csv関数のテスト
    print("==== extract_latlngArray_csvのテスト ====")
    sample_file_path = 'sample/warabi_hinansisetu.csv'
    lat_array_sample, lng_array_sample = extract_latlngArray_csv(filepath=sample_file_path)
    print("file:", sample_file_path)
    print("latitude array:",lat_array_sample)
    print("longitude array:", lng_array_sample)
    print("¥n")


    # extract_latlngBbox_csv関数のテスト
    print("==== extract_latlngBbox_csvのテスト ====")
    sample_file_path = 'sample/warabi_hinansisetu.csv'
    east_sample, west_sample, north_sample, south_sample = extract_latlngBbox_csv(filepath=sampl
e_file_path)
    print("file:", sample_file_path)
    print("east, west, north, south:",east_sample, west_sample, north_sample, south_sample)
    print("¥n")
```

```
==== extract_latlngArray_csvのテスト ====
file: sample/warabi_hinansisetu.csv
latitude array: [35.82070833 35.81930833 35.82445639 35.82534444 35.82696667 35.82
767778
 35.82341389 35.82967222 35.82363333 35.82146528 35.82082222 35.8226
 35.81826667 35.81517778 35.81342222 35.81494167 35.81787222 35.81504444
 35.81626944 35.82188611 35.82177778 35.82026111 35.82109722 35.82259722
 35.81621417 35.8259     35.8231     35.8196    ]
longitude array: [139.6777    139.6763611 139.6768878 139.6756306 139.6712556 139.
6838
 139.6820083 139.6774167 139.6943306 139.6906711 139.6869056 139.6852139
 139.68665   139.6909667 139.6913806 139.6955028 139.7009028 139.6994111
 139.7020778 139.7026167 139.704475  139.7035667 139.7078028 139.7060167
 139.6886544 139.681     139.6843    139.708    ]


==== extract_latlngBbox_csvのテスト ====
file: sample/warabi_hinansisetu.csv
east, west, north, south: 139.708 139.6712556 35.82967222 35.81342222
```

In [7]:

```
# FindByKeyword動作テスト1
print("==== FindByKeywordのテスト1 ====")
locationKeyword_sample1 = '羽田空港'
g1 = FindByKeyword(locationKeyword_sample1)
print("Keyword: ", locationKeyword_sample1)
for elem in g1:
    print(
        elem.geonames_id,
        elem.address,
        elem.country,
        elem.state,
        elem.lat,
        elem.lng,
        elem.feature_class, )
print("¥n")

# FindByKeyword動作テスト2
print("==== FindByKeywordのテスト1 ====")
locationKeyword_sample2='中央区' # 中央区
g2 = FindByKeyword(locationKeyword_sample2,featureClass=['A','P'])
print("Keyword: ", locationKeyword_sample2)
for elem in g2:
    print(
        elem.geonames_id,
        elem.address,
        elem.country,
        elem.state,
        elem.lat,
        elem.lng,
        elem.feature_class, )
print("¥n")
```

```
==== FindByKeywordのテスト1 ====
Keyword:  羽田空港
6300412 羽田空港 日本 東京都 35.55226 139.77969 S
10957423 羽田空港 日本 東京都 35.55281 139.77933 L


==== FindByKeywordのテスト1 ====
Keyword:  中央区
1859171 神戸 日本 兵庫県 34.6913 135.183 P
2113015 千葉市 日本 千葉県 35.6 140.11667 P
1858421 熊本 日本 熊本県 32.80589 130.69181 P
1848254 Yono 日本 埼玉県 35.88333 139.63333 P
1864487 中央区 日本 東京都 35.66993 139.77705 A
1863623 銀座 日本 東京都 35.67184 139.76717 P
1849687 月島 日本 東京都 35.66259 139.78195 P
8534447 中央区 日本 千葉県 35.58434 140.12941 A
7391689 中央区 日本 大阪府 34.68011 135.51413 A
8534438 中央区 日本 埼玉県 35.88133 139.62468 A
```

In [8]:

```
# GetFeatures関数の動作テスト
print("==== GetFeaturesのテスト ====")

geonameId_sample=2113015
result_elem = getFeautures(geonameId=geonameId_sample)

print("geonameId:", geonameId_sample)
print(
    result_elem['geonameId'],
    result_elem['name'],
    result_elem['countryName'],
    result_elem['adminName1'],
    result_elem['lat'],
    result_elem['lng'],
    result_elem['fcl'],
    result_elem['bbox']
)
print("¥n")
```

```
==== GetFeaturesのテスト ====
geonameId: 2113015
2113015 千葉市 日本 千葉県 35.6 140.11667 P {'east': 140.26997079306582, 'south':
35.47554271828871, 'north': 35.724457281711295, 'west': 139.9633632069342, 'accura
cyLevel': 0}
```

In [9]:

```
# FindByBbox関数のテスト
print("==== FindByBboxのテスト ====")
east_sample=141.0
west_sample=139.9
north_sample=35.8
south_sample=35.4
g3 = FindbyBbox(east=east_sample,west=west_sample,north=north_sample,south=south_sample)
print("east,west,north,south:", east_sample,west_sample,north_sample,south_sample)
for elem in g3:
    print(
        elem.geonames_id,
        elem.address,
        elem.country,
        elem.state,
        elem.lat,
        elem.lng,
        elem.feature_class, )
print("¥n")
```

```
==== FindByBboxのテスト ====
east,west,north,south: 141.0 139.9 35.8 35.4
2113015 千葉市 日本 千葉県 35.6 140.11667 P
1863905 本町 日本 千葉県 35.70129 139.98648 P
2113014 千葉県 日本 千葉県 35.60506 140.12333 A
2111684 成田 日本 千葉県 35.78333 140.31667 P
2112664 市原市 日本 千葉県 35.51667 140.08333 P
1857553 松戸市 日本 千葉県 35.77995 139.90144 P
2110579 八街市 日本 千葉県 35.65 140.31667 P
2111220 佐倉 日本 千葉県 35.71667 140.23333 P
2111855 モバラ 日本 千葉県 35.42583 140.29608 P
2110480 四街道市 日本 千葉県 35.65 140.16667 P
```

In [10]:

```
# FindNearby関数の動作テスト
print("==== FindNearbyのテスト ====")
lat_sample = 35.70552
lng_sample = 139.46125
result = FindNearby(lat=lat_sample,lng=lng_sample)
print("lat,lng:", lat_sample,lng_sample)
for elem in result:
    print(
        elem['geonameId'],
        elem['name'],
        elem['countryName'],
        elem['adminName1'],
        elem['lat'],
        elem['lng'],
        elem['fcl'],
        elem['distance']
    )
print("¥n")
```

```
==== FindNearbyのテスト ====
lat,lng: 35.70552 139.46125
1858962 国分寺市 日本 東京都 35.70552 139.46125 A 0.00024
7424846 こくぶんじしやくしょ 日本 東京都 35.71083 139.46222 S 0.59602
7551898 こいがくぼえき 日本 東京都 35.71135 139.46389 S 0.68957
7551914 にしこくぶんじえき 日本 東京都 35.69991 139.46598 S 0.75569
1858964 国分寺 日本 東京都 35.70222 139.47556 P 1.34536
7551870 くにたちえき 日本 東京都 35.6992 139.44642 S 1.51494
11790983 上水本町 日本 東京都 35.71436 139.47423 P 1.53026
11807449 Izumicho 日本 東京都 35.69805 139.4759 P 1.5636
11807172 西元町 日本 東京都 35.69362 139.47118 P 1.59718
1859265 Kita-tama-gun 日本  35.71667 139.45 A 1.60187
```

In [11]:

```
    # データカタログのタイトル、説明文から地名キーワードを抽出し、地名候補を出力
    print("==== データカタログのタイトル、説明文から地名キーワードを抽出し、地名候補を出力 ====")
)
    title_sample = "八王子市の赤ちゃん・ふらっと設置施設"
    description_sample = "【八王子市】子育て関連オープンデータ"
    # https://catalog.data.metro.tokyo.lg.jp/dataset/t132012d0000000031

    locationKeyword_list = extract_LocationKeywords(title_sample+description_sample)
    locationKeyword_sample = locationKeyword_list[0]

    g5 = FindByKeyword(locationKeyword_sample)
    print("title:",title_sample)
    print("description:", description_sample)
    print("Keyword:", locationKeyword_sample)
    for elem in g5:
        print(
            elem.geonames_id,
            elem.address,
            elem.country,
            elem.state,
            elem.lat,
            elem.lng,
            elem.feature_class
            )
    print("¥n")
```

```
==== データカタログのタイトル、説明文から地名キーワードを抽出し、地名候補を出力 ==
==
title: 八王子市の赤ちゃん・ふらっと設置施設
description: 【八王子市】子育て関連オープンデータ
Keyword: 八王子市
1863440 八王子 日本 東京都 35.65583 139.32389 P
8304375 八王子市 日本 東京都 35.65983 139.28766 A
1857416 Meijinomori-Takao-kokutei-kōen 日本 東京都 35.61667 139.25 L
1859653 Kawaranoshuku 日本 東京都 35.64139 139.27472 P
1907169 Matsugi 日本 東京都 35.62056 139.38972 P
11790344 日吉町 日本 東京都 35.66101 139.31409 P
7551575 高尾山口駅 日本 東京都 35.63238 139.26981 S
7551617 めじろだいえき 日本 東京都 35.64364 139.30851 S
7551676 京王八王子駅 日本 東京都 35.65805 139.34298 S
7551717 こみやえき 日本 東京都 35.68586 139.36848 S
```

In [12]:

```
    # ファイルからboundingboxを算出し、そのエリアにある地名候補を出力
    print("==== ファイル取得から地名候補出力までのテスト ====")
    sample_file_path = 'sample/warabi_hinansisetu.csv'
    east_sample, west_sample, north_sample, south_sample = extract_latlngBbox_csv(filepath=sampl
e_file_path)
    g6 = FindbyBbox(east=east_sample,west=west_sample,north=north_sample,south=south_sample)
    print("File:", sample_file_path)
    print("east,west,north,south:", east_sample,west_sample,north_sample,south_sample)
    for elem in g6:
        print(
            elem.geonames_id,
            elem.address,
            elem.country,
            elem.state,
            elem.lat,
            elem.lng,
            elem.feature_class, )
```

```
==== ファイル取得から地名候補出力までのテスト ====
File: sample/warabi_hinansisetu.csv
east,west,north,south: 139.708 139.6712556 35.82967222 35.81342222
1907301 シモトダ 日本 埼玉県 35.815 139.6853 P
1848902 蕨市 日本 埼玉県 35.82526 139.6855 A
8572571 きたまち 日本 埼玉県 35.82935 139.68181 P
8572572 にしきちょう 日本 埼玉県 35.82545 139.67181 P
11032606 カミトダ 日本 埼玉県 35.81706 139.67576 P
11032713 つかごし 日本 埼玉県 35.82378 139.69996 P
11032714 ミナミチョウ 日本 埼玉県 35.81807 139.69442 P
11032743 しばしんまち 日本 埼玉県 35.82938 139.69558 P
11032744 芝中田 日本 埼玉県 35.82935 139.70099 P
11612339 蕨 日本 埼玉県 35.82188 139.68545 P
```